Home NIDS

ระบบตรวจจับการบุกรุกเครือข่ายบ้าน ด้วย Machine Learning (Anomaly-based) · คู่มือฉบับย่อ

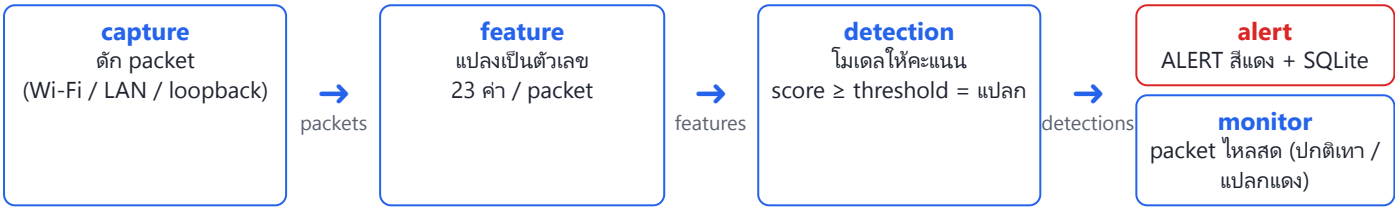
Python · Scapy · ZeroMQ · Isolation Forest
โฟลเดอร์: `C:\Project_AI\home-nids`

■ ภาพรวม / การทำงาน ■ โครงสร้างไฟล์ ■ วิธีใช้งาน ■ ข้อควรระวัง ■ การเทรนโมเดล

1 เป้าหมาย และ ระบบทำงานอย่างไร

เป้าหมาย: รันระบบค้างไว้บนเครื่องที่รับ traffic จาก switch (SPAN) แล้วเมื่อมีสิ่งผิดปกติ เช่น `ping` หรือ `nmap` ต้องขึ้น **ALERT สีแดง** ภายในไม่กี่วินาที และบันทึกลงฐานข้อมูล โดยเขียนเองทั้งหมด ไม่ใช้ IDS สำเร็จรูป (Snort, Suricata, Zeek)

หลักคิด (Anomaly-based): โมเดลเรียนรู้แค่ว่า traffic ปกติ หน้าตาเป็นอย่างไร อะไรที่ต่างจากนั้นมากจะถูกมองว่า "แปลก" ไม่ต้องมีตัวอย่างการโจมตีตอนเทรน



แต่ละกล่อง = โปรแกรมแยกกัน (microservice) คุยกันผ่าน ZeroMQ ด้วย JSON · ตั้งค่าทั้งหมดในไฟล์เดียว `config/config.yaml`

โมเดลดูอะไรใน packet (feature)

ต่อ packet	protocol (TCP/UDP/ICMP), ขนาด, TTL, TCP flags (SYN/ACK/FIN/...), ICMP type, กลุ่มของ port ปลายทาง
บริบท 5 วินาทีล่าสุดของเครื่องต้นทาง	ส่งกี่ packet · ไปกี่ port · ไปกี่เครื่อง · ไปกี่ port ของ service (< 49152) · สัดส่วนเครื่องปลายทางต่อ packet → ทำให้จับการสแกน (nmap) ได้ เพราะตัวสแกนยิงหลาย port/หลายเครื่องในเวลาสั้น

2 โฟลเดอร์และไฟล์

ที่อยู่	ความหมาย
🟢 **ส่วนที่คุณใช้ (อยู่ชั้นนอก)**	
`config/config.yaml`	**ไฟล์ตั้งค่าเดียวของระบบ**: interface ที่จะดัก, threshold, cooldown, path ของ pcap
`training/`	**`train.ipynb` = ตัวเทรนหลัก (Jupyter)** · `pcap/normal/` วาง pcap ปกติ · `pcap/test/` วาง pcap ที่มีการโจมตี · `pcap/labels.csv` กฎ label · (`train.py`, `pcap_to_csv.py`, `evaluate.py` = โค้ดเบื้องหลัง notebook)
`data/`	ข้อมูลเทรน (.csv), pcap ที่แปลงแล้ว (`pcap_csv/`), **`alerts.db` = ประวัติ alert**
`run_all.py`	**รันทั้งระบบ** (`.ps1` Windows, `.sh` Linux)
`README.md` / `.pdf`	เอกสารสำหรับ AI / คู่มือนี้ · `requirements.txt` ใช้ตอนติดตั้ง · `pytest.ini` ตั้งค่าการทดสอบ
🔵 **`system/` — ระบบใช้เอง ไม่ต้องแก้**	
`system/services/`	5 โปรแกรมหลัก: capture → feature → detection → alert / monitor
`system/shared/`	โค้ดใช้ร่วมกัน · `features.py` = สูตรแปลง packet เป็นตัวเลข (ใช้ทั้งเทรนและรันจริง)
`system/models/`	โมเดลที่เทรนแล้ว (notebook บันทึกให้เอง) — **ห้ามลบ**
`system/tests/`	ชุดทดสอบ (รัน `python -m pytest` จากโฟลเดอร์หลัก)
`system/scripts/`	`list_interfaces.py` ดูชื่อ network interface
`system/docs/`	ต้นฉบับ HTML ของ README.pdf

3 วิธีใช้งาน

① ติดตั้ง (ครั้งแรกครั้งเดียว)

```
pip install -r requirements.txt
```

Windows: ลง **Npcap** และติ๊ก *Support loopback traffic* · Linux: ลง libpcap

② เลือก interface ที่จะดัก

```
python system/scripts/list_interfaces.py
```

ใส่ชื่อใน `config.yaml → capture.interfaces` เช่น `"Wi-Fi"`, `"loopback"`
ใช้ SPAN: เสียบสาย LAN เข้าพอร์ตที่ mirror แล้วใส่ `"Ethernet"`

③ รันระบบ (PowerShell แบบ Administrator)

```
cd C:\Project_AI\home-nids
python run_all.py
```

หน้าต่างนี้แสดง ALERT · หน้าต่าง monitor เปิดแยกเอง · ปิดด้วย **Ctrl+C**

④ ทดสอบ (เปิด terminal อีกอัน)

```
ping 172.20.10.2       # IP เครื่อง (ดู ipconfig)
nmap -sS 172.20.10.2   # ใช้ IP จริง ไม่ใช่ 127.0.0.1
```

ผลที่ควรเห็น:

```
ALERT 12:34:56  172.20.10.2 → 172.20.10.2  ICMP
   เหตุผล: ICMP echo request (ping) ไม่พบใน traffic ปกติ  score=0.717
```

⑤ ดูประวัติ alert

```
python -c "import sqlite3; [print(r) for r in sqlite3.connect('data/alerts.db').execute('select src_ip,dst_ip,proto,count,reason from alerts')]"
```

⑥ ตรวจว่าโค้ดยังทำงานถูก

```
python -m pytest
```

4 ข้อควรระวัง และปัญหาที่พบบ่อย

อาการ	แก้ไข
ไม่พบไฟล์โมเดล	รัน `generate_data.py` แล้ว `train.py`
ไม่พบ interface	ดูชื่อจาก `system/scripts/list_interfaces.py`
ping ตัวเองไม่ขึ้น	ต้องมี `"loopback"` ใน config + Npcap loopback
ดัก packet ไม่ได้	เปิด terminal แบบ Administrator / ใช้ sudo
feature ไม่ตรงกับโมเดล	แก้ `features.py` แล้วต้องเทรนใหม่

- รองรับเฉพาะ **IPv4**
- การสแกนที่**ช้ามาก** (nmap `-T0/-T1`) หลุดได้ เพราะดูทีละ 5 วินาที
- ยังไม่ได้ทดสอบกับ nmap ตัวจริง, ping จากเครื่องอื่น และ SPAN จริง
- เครื่องนี้ **ห้ามใช้ pandas** (Windows บล็อก) โปรเจกต์จึงใช้ numpy + csv แทน
- TCP ระหว่าง 127.0.0.1 ↔ 127.0.0.1 ถูกข้ามโดยตั้งใจ (เป็น traffic ภายในของระบบเอง)

5 การเทรนโมเดลด้วย Jupyter (ใช้ dataset จากออนไลน์ / Wireshark)

หลักสำคัญ: เทรนด้วย **traffic ปกติล้วน** (ไม่ต้องมี label) — ส่วนข้อมูลที่ **มีการโจมตี + label** ใช้แค่ **วัดผล** (TP/FP/TN/FN) ถ้ามีการโจมตีปนในข้อมูลเทรน โมเดลจะเรียนว่าการโจมตีนั้นปกติ

ขั้นตอน (ทำครั้งเดียวจบใน notebook)

① โหลด dataset
เลือกชุดที่มี `.pcap` เช่น CIC-IDS2017

② วางไฟล์
ปกติ → `training/pcap/normal/`
มีการโจมตี → `training/pcap/test/`

③ เขียน label
ใส่ IP ผู้โจมตี + ช่วงเวลาใน `training/pcap/labels.csv`

④ เปิด notebook
`python -m notebook training/train.ipynb` แล้วกด **Run All**

⑤ อ่านผล
Confusion Matrix + Accuracy, Precision, Recall, F1

⑥ ใช้งาน
โมเดลถูกบันทึกลง `system/models/` แล้ว → รัน `run_all.py` ใหม่

สิ่งที่ notebook ทำให้อัตโนมัติ

ขั้น	ทำอะไร
0	อ่าน parameter จาก `config.yaml` + ใช้โค้ดของ microservice (สูตร feature ตัวเดียวกับระบบจริง)
1	แปลง pcap → CSV (เก็บไว้ที่ `data/pcap_csv/` ครั้งต่อไปไม่ต้องแปลงซ้ำ)
2–4	รวมข้อมูลปกติ ตัด ping ออก ดูกราฟ แล้วเทรน
5	สร้างชุดทดสอบ: ไฟล์ใน `test/` + การโจมตีจำลอง (ping, nmap)
6–7	เลือก threshold แล้ววัดผล พร้อมกราฟ Confusion Matrix / ROC
8	บันทึกโมเดล + threshold ลง `system/models/`

ตัวอย่าง: CIC-IDS2017

ไฟล์	วางที่
`Monday-WorkingHours.pcap`	`normal/` (วันปกติ)
`Friday-WorkingHours.pcap`	`test/` (มี PortScan)

`labels.csv` (เอา `#` หน้าบรรทัดตัวอย่างออก):

```
PortScan,172.16.0.1,192.168.10.50,2017-07-07T13:55:00-03:00,2017-07-07T14:35:00-03:00,0,CICFriday
```

ตรวจ IP/เวลากับหน้าเว็บ dataset อีกครั้ง · ไฟล์ใหญ่มาก อ่านแค่ `max_packets_per_pcap` (2 ล้าน) ต่อไฟล์

อ่านผลการวัด

ค่า	ความหมาย
TP / FN	การโจมตีที่ จับได้ / หลุด
FP / TN	ปกติแต่ **alert หลอก** / ปกติและตอบถูก
Accuracy	ตอบถูกกี่ % (⚠️ หลอกตาเมื่อข้อมูลปกติเยอะมาก)
Precision	alert ที่ขึ้น เป็นการโจมตีจริงกี่ %
Recall	การโจมตีทั้งหมด จับได้กี่ %
F1	ค่าเฉลี่ยของ Precision กับ Recall
FPR	traffic ปกติ ถูก alert หลอกกี่ %
ROC-AUC	แยกปกติ/โจมตีได้ดีแค่ไหน (1.0 = สมบูรณ์)

ผลปัจจุบัน (ข้อมูลจำลอง, threshold 0.6308)

Accuracy	Precision	Recall	F1	FPR	AUC
0.966	0.986	0.909	0.946	0.006	0.991

ตั้งค่าในเซลล์แรกของ notebook

ค่า	ใช้ทำอะไร
`USE_SYNTHETIC`	รวมข้อมูลจำลองไปด้วย (แนะนำ True)
`DROP_ICMP_ECHO`	ตัด ping ออกจากข้อมูลเทรน
`USE_SIM_TEST`	เพิ่ม ping/nmap จำลองในการวัดผล
`THRESHOLD_MODE`	`"recommended"` (แนะนำ) · `"best_f1"` (ตัวเลขสวยเกินจริง) · หรือตัวเลข

แหล่งข้อมูลอื่นที่ใช้ได้: ไฟล์ที่ **Save จาก Wireshark** (วางใน `normal/` ได้เลย) · บันทึกจากบ้านด้วย `capture.mode: csv` · ใช้ไม่ได้: CSV แบบ flow (Kaggle, NSL-KDD) เพราะระบบเราทำงานระดับ packet · เทรนแบบ command line: `python training/train.py`